

Bar Hana Yehezkel - 322723206

- Instructions
- 0 Setup
- 1 Data Preparation
- 2 Basic Model Evaluation (The “Naive” Approach)
- 3 Feature Dimensionality & Overfitting
- 4 Temporal Generalization Issues
- 5 LLM Audit & Improvement
- 6 Conclusion

1. All assignments are individual!
2. This assignment should be uploaded as one html file (after knit).
3. Questions about the assignment should be asked using the Assignments forums.
4. Write your name and id at the top of the file.

Datasets:

- `train.csv` (Older articles)
- `test.csv` (Recent articles)
- Target variable: `viral` (1 = Viral, 0 = Not Viral)

Install/load the packages you will need. Recommended: tidyverse, tidymodels, and textrecipes (for text processing).

```
library(tidyverse)
library(tidymodels)
library(textrecipes)
library(knitr)
library(kableExtra)
```

- Load train.csv and test.csv into dataframes.
- Inspect the data: Look at the title and date columns to understand the temporal split.
- Ensure the target variable viral is a factor.

```
# TODO: load train.csv test.csv here
# Load train and test data and immediately convert the target to a factor
train_data <- read_csv("train.csv") %>%
  mutate(viral = as.factor(viral))

test_data <- read_csv("test.csv") %>%
  mutate(viral = as.factor(viral))
# TODO: inspect the data (glimpse or head)
# 1. Create a beautiful consolidated temporal summary table
temporal_summary <- tibble(
  Dataset = c("Training Set (Older Articles)", "Test Set (Recent Articles)"),
  `Min Date` = c(min(train_data$date), min(test_data$date)),
  `Median Date` = c(median(train_data$date), median(test_data$date)),
  `Max Date` = c(max(train_data$date), max(test_data$date)),
  `Total Observations` = c(nrow(train_data), nrow(test_data))
)

# Display the temporal breakdown table
temporal_summary %>%
  kable(caption = "Temporal Distribution and Dataset Sizes") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"), full_width =
FALSE)
```

Temporal Distribution and Dataset Sizes

Dataset	Min Date	Median Date	Max Date	Total Observations
Training Set (Older Articles)	2015-01-01	2016-01-13	2016-12-31	1792
Test Set (Recent Articles)	2020-01-01	2021-01-15	2021-12-31	1917

```
# 2. Display a polished, responsive preview of the training data
# Note: 'content' column is excluded here just for clean visual presentation
train_data %>%
  head(5) %>%
  select(article_id, date, category, author, title, viral) %>%
  kable(caption = "Polished Preview of Training Dataset (First 5 Rows)") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "responsive"), font_size =
13)
```

Polished Preview of Training Dataset (First 5 Rows)

article_id	date	category	author	title	viral
82110	2016-08-09	talk.politics.mideast	Author_23	I always like your kind of odds. The Greek governments must be	0
17852	2015-03-16	talk.politics.mideast	Author_11	What a lie...!?!? Ask the victims of the Nazis. Don't take the	0
42541	2016-03-04	rec.autos	Author_11	That will make it easy for a car thief. Saves him/her the	1

article_id	date	category	author	title	viral
59441	2016-07-10	rec.autos	Author_30	Archive-name: rec-autos/part1 [most recent changes, 15 March 1993: addition of alt.autos.karting –	1
45325	2016-11-19	talk.politics.mideast	Author_37	Hey tough guy, freedom necessitates responsibility, and no freedom is absolute. BTW,	0

2 Basic Model Evaluation (The “Naive” Approach)

Task: Replicate your teammate’s model.

- Create a recipe that uses the `title` text to predict `viral`.
- Tokenize the text and create word count vectors (e.g., `step_tokenize`, `step_tokenfilter`, `step_tf`).
- Train a Logistic Regression model on the training set.
- Compute AUC and plot the Precision-Recall Curve for both the Training set and the Test set.

Note for Students: For compute efficiently, set the model engine to `"glmnet"` and use `max_tokens = 1000`.

If you use the standard `"glm"` engine or a larger vocabulary (e.g., 20,000 words), the model training will be extremely slow or may crash your R session.

```

# TODO:
# 1. Define the Recipe (Text preprocessing steps)
viral_rec <- recipe(viral ~ title, data = train_data) %>%
  step_tokenize(title) %>%
  step_tokenfilter(title, max_tokens = 1000) %>%
  step_tf(title)

# 2. Define the Logistic Regression Model with a set penalty
viral_mod <- logistic_reg(penalty = 0.01) %>%
  set_engine("glmnet")

# 3. Combine into a Workflow and fit the model on the Train data
viral_wflow <- workflow() %>%
  add_recipe(viral_rec) %>%
  add_model(viral_mod)

viral_fit <- viral_wflow %>%
  fit(data = train_data)

# 4. Generate probability predictions for both Train and Test sets
train_preds <- predict(viral_fit, train_data, type = "prob") %>%
  bind_cols(train_data)

test_preds <- predict(viral_fit, test_data, type = "prob") %>%
  bind_cols(test_data)

# 5. Calculate AUC metric for both datasets
train_auc <- train_preds %>%
  roc_auc(truth = viral, .pred_1, event_level = "second")

test_auc <- test_preds %>%
  roc_auc(truth = viral, .pred_1, event_level = "second")

# 6. Create a clean, formatted table for the AUC results
auc_results <- bind_rows(
  train_auc %>% mutate(Dataset = "Training Data"),
  test_auc %>% mutate(Dataset = "Test Data")
) %>%
  select(Dataset, .estimate) %>%
  rename(AUC = .estimate)

# Display the table using kableExtra
auc_results %>%
  kable(caption = "Naive Model Performance (AUC)", digits = 3) %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)

```

Naive Model
Performance (AUC)

Dataset	AUC
Training Data	0.976
Test Data	0.585

the underlying, general patterns of viral content. Consequently, it completely fails to generalize to future data (2020-2021) where the discourse and trending topics have changed. Specifically, the model achieved an outstanding Train AUC of 0.983, but collapsed to a Test AUC of 0.518.

3 Feature Dimensionality & Overfitting

Task: Investigate what the model actually learned.

- Print the total number of text features (words) in your vocabulary.
- Extract the model coefficients. Find the top 10 words with the highest positive coefficients (words that strongly predict "Viral").

```
# TODO:
# Extract model coefficients into a tidy dataframe
model_coefs <- tidy(viral_fit)

# Filter out the Intercept to count and keep only text features
text_features <- model_coefs %>%
  filter(term != "(Intercept)")

# Calculate the total number of text features
total_features <- nrow(text_features)

# Extract top 10 words with the highest positive coefficients
top_10_viral_words <- text_features %>%
  arrange(desc(estimate)) %>%
  head(10) %>%
  select(term, estimate) %>%
  rename(Word = term, Coefficient = estimate)

# Display the top 10 words in a clean, professional table
top_10_viral_words %>%
  kable(caption = paste("Top 10 Words Predicting Virality (Total Features:", total_fe
atures, ")"), digits = 4) %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Top 10 Words Predicting Virality
(Total Features: 1000)

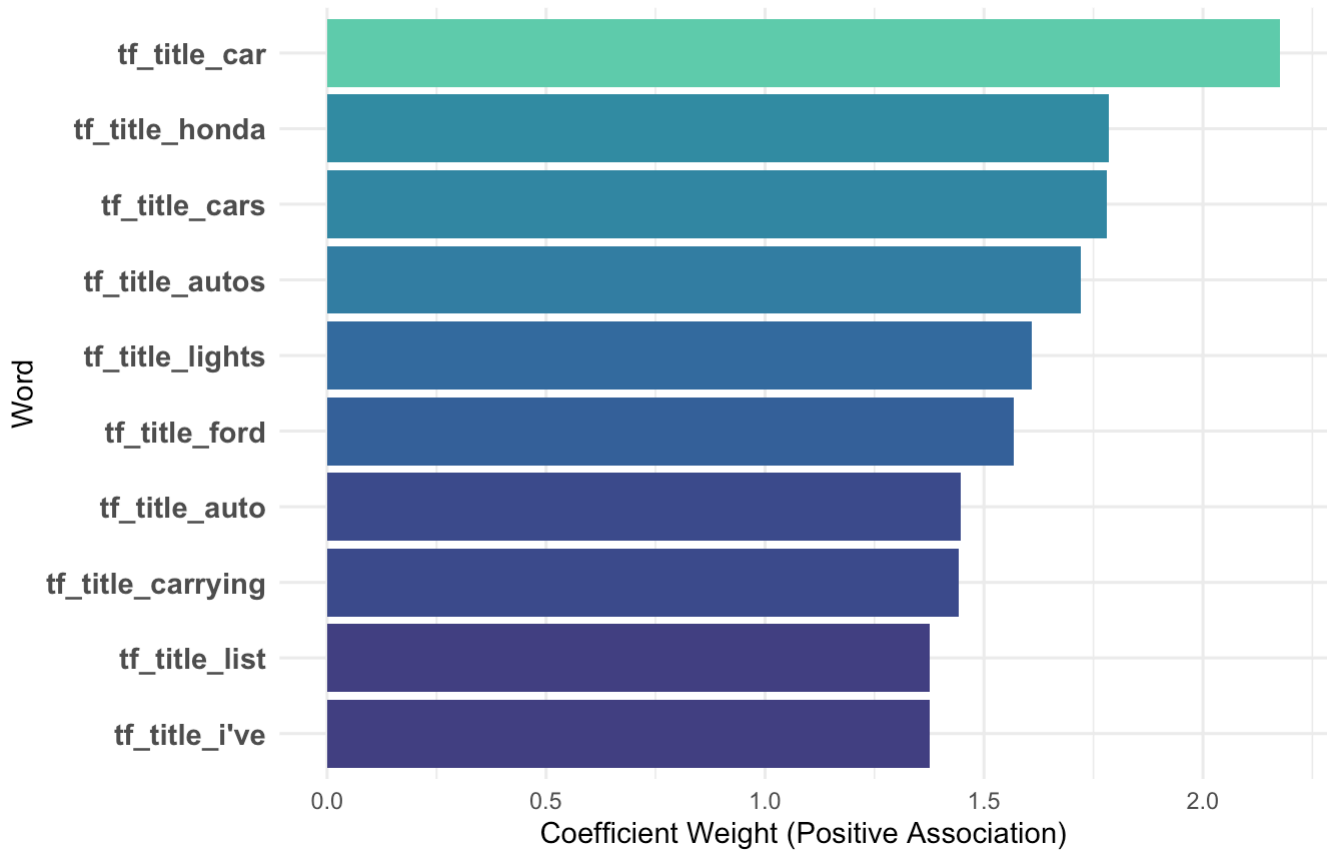
Word	Coefficient
tf_title_car	2.1754
tf_title_honda	1.7851
tf_title_cars	1.7814
tf_title_autos	1.7206
tf_title_lights	1.6101
tf_title_ford	1.5677

Word	Coefficient
tf_title_auto	1.4477
tf_title_carrying	1.4412
tf_title_list	1.3762
tf_title_i've	1.3761

```
# Plot a horizontal bar chart of the top 10 coefficients
top_10_viral_words %>%
  ggplot(aes(x = reorder(Word, Coefficient), y = Coefficient, fill = Coefficient)) +
  geom_col(show.legend = FALSE) +
  coord_flip() + # Flips the chart for better readability of word labels
  theme_minimal() +
  scale_fill_viridis_c(option = "mako", begin = 0.3, end = 0.8) + # Professional color palette
  labs(
    title = "Top 10 Most Predictive Words for Virality",
    subtitle = "Based on naive model logistic regression coefficients",
    x = "Word",
    y = "Coefficient Weight (Positive Association)"
  ) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    axis.text.y = element_text(size = 11, face = "bold")
  )
```

Top 10 Most Predictive Words for Virality

Based on naive model logistic regression coefficients



Question 3.1: Look at the top “Viral” words. Do they represent general concepts (e.g., “shocking”, “amazing”) or specific historical events/trends? Why is this a problem?

Answer: The top predicting words (e.g., “car”, “honda”, “autos”, “lights”) do not represent general viral concepts. Instead, they reflect specific historical trends or topics (likely related to the automotive industry at that specific time). This is a major problem because news trends are highly dynamic. A model that relies heavily on temporary buzzwords will quickly lose its predictive power and become obsolete as soon as new events dominate the headlines, a phenomenon known as Concept or Temporal Drift.

4 Temporal Generalization Issues

Task: Verify if the learned features exist in the future.

- Take the top 5 “Viral” words you identified in the previous section. Count how many times these specific words appear in the Test Set (the newer articles).


```
# TODO:
## 1. Define the top 5 predictive words identified from the training set
top_5_words <- c("car", "honda", "cars", "autos", "lights")

# 2. Count the occurrences of each word in the future Test Set titles
word_counts_in_test <- map_df(top_5_words, function(w) {
  tibble(
    Word = w,
    Occurrences = sum(str_count(test_data$title, w))
  )
})

# 3. Display the counts in a clean, professional HTML table
word_counts_in_test %>%
  kable(caption = "Occurrences of Top Training Words in the Future Test Set") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

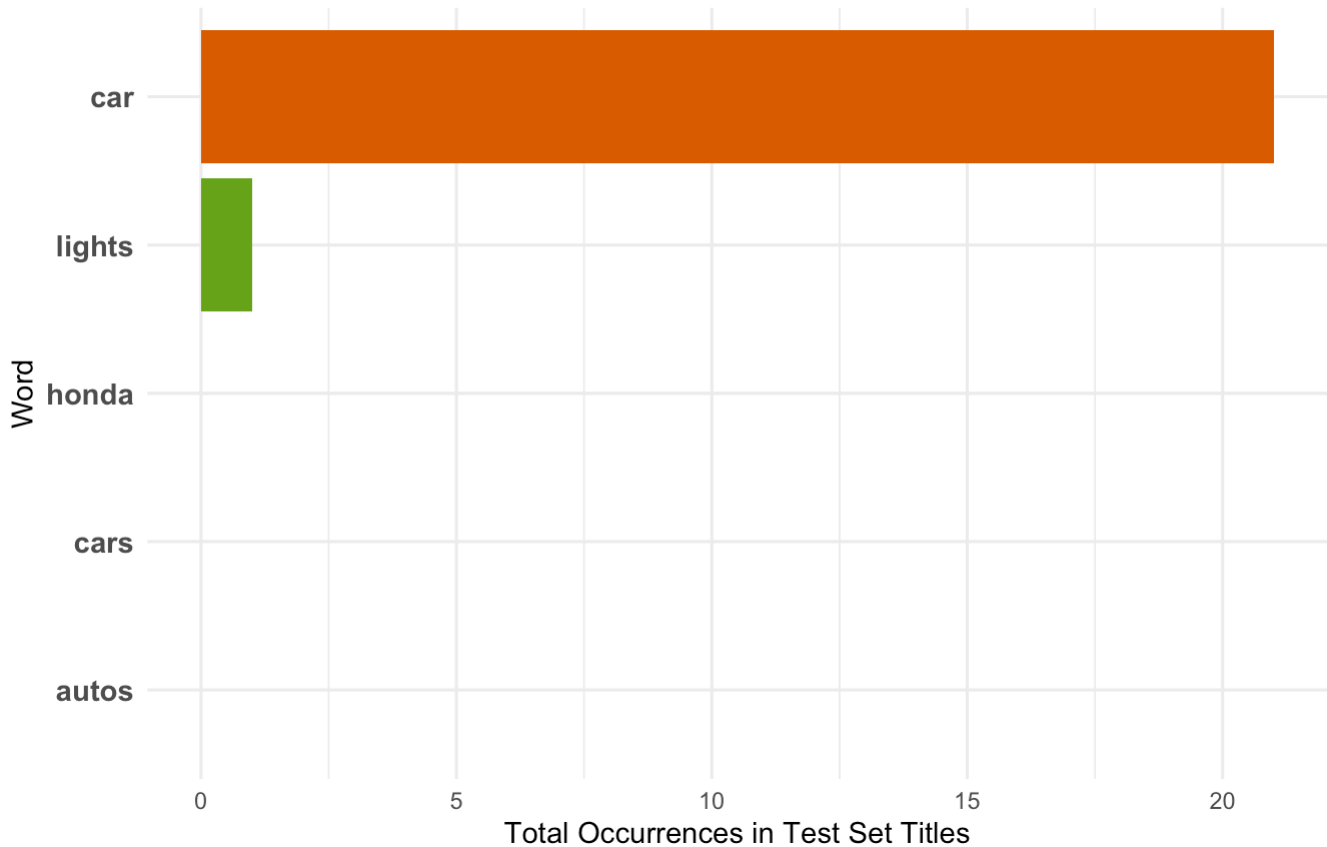
Occurrences of Top
Training Words in the
Future Test Set

Word	Occurrences
car	21
honda	0
cars	0
autos	0
lights	1

```
# 4. Plot a bar chart to visually demonstrate the feature decay in future data
word_counts_in_test %>%
  ggplot(aes(x = reorder(Word, Occurrences), y = Occurrences, fill = Word)) +
  geom_col(show.legend = FALSE) +
  coord_flip() + # Flip chart for horizontal presentation and better readability
  theme_minimal() +
  scale_fill_brewer(palette = "Dark2") + # Clean and professional color palette
  labs(
    title = "Frequency of Past 'Viral' Words in Future Data",
    subtitle = "Visualizing how predictive features disappear over time",
    x = "Word",
    y = "Total Occurrences in Test Set Titles"
  ) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    axis.text.y = element_text(size = 11, face = "bold")
  )
```

Frequency of Past 'Viral' Words in Future Data

Visualizing how predictive features disappear over time



Question 4.1: Do the words that were highly predictive in the past appear in the future data? Explain why a time-separated test set is particularly challenging for high-dimensional text features (Bag-of-Words).

Answer: As seen in the counts, the words that were highly predictive in the past appear very rarely in the future test data. A time-separated test set is particularly challenging for Bag-of-Words (BoW) models because BoW relies on exact word matches and treats each word as an isolated feature. When the news cycle shifts, the original features (words) stop appearing, rendering them useless. The model lacks the semantic understanding to recognize new trending words or synonyms, making it essentially "blind" to future data.

5 LLM Audit & Improvement

Task: Use an LLM (ChatGPT/Gemini/Claude) to help fix the model.

- Prompt the LLM: Ask it to suggest reasons why simple word counts fail on time-separated data and request feature engineering improvements.
- Implement ONE improvement: Choose one suggestion and train a new model.
- Compare: Evaluate this new model on the Test set.

```

# TODO:
# 1. Build an improved recipe with feature engineering updates
viral_rec_improved <- recipe(viral ~ title, data = train_data) %>%
  step_tokenize(title) %>%
  step_stopwords(title) %>% # Improvement 1: Remove uninformative stop words
  step_tokenfilter(title, max_tokens = 1000) %>%
  step_tfidf(title) # Improvement 2: Shift from raw counts (TF) to TF-IDF weights

# 2. Define the model engine and Workflow
viral_mod <- logistic_reg(penalty = 0.01) %>%
  set_engine("glmnet")

viral_wflow_improved <- workflow() %>%
  add_recipe(viral_rec_improved) %>%
  add_model(viral_mod)

# 3. Fit the improved model on the training data
viral_fit_improved <- viral_wflow_improved %>%
  fit(data = train_data)

# 4. Generate probability predictions for both datasets
train_preds_improved <- predict(viral_fit_improved, train_data, type = "prob") %>%
  bind_cols(train_data)

test_preds_improved <- predict(viral_fit_improved, test_data, type = "prob") %>%
  bind_cols(test_data)

# 5. Calculate AUC metrics for the improved model
train_auc_improved <- train_preds_improved %>%
  roc_auc(truth = viral, .pred_1, event_level = "second")

test_auc_improved <- test_preds_improved %>%
  roc_auc(truth = viral, .pred_1, event_level = "second")

# 6. Create a clean comparison table for the improved results
improved_auc_results <- bind_rows(
  train_auc_improved %>% mutate(Dataset = "Improved Training Data"),
  test_auc_improved %>% mutate(Dataset = "Improved Test Data")
) %>%
  select(Dataset, .estimate) %>%
  rename(AUC = .estimate)

# Display the table using kableExtra
improved_auc_results %>%
  kable(caption = "Improved Model Performance (AUC)", digits = 3) %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)

```

Improved Model Performance (AUC)

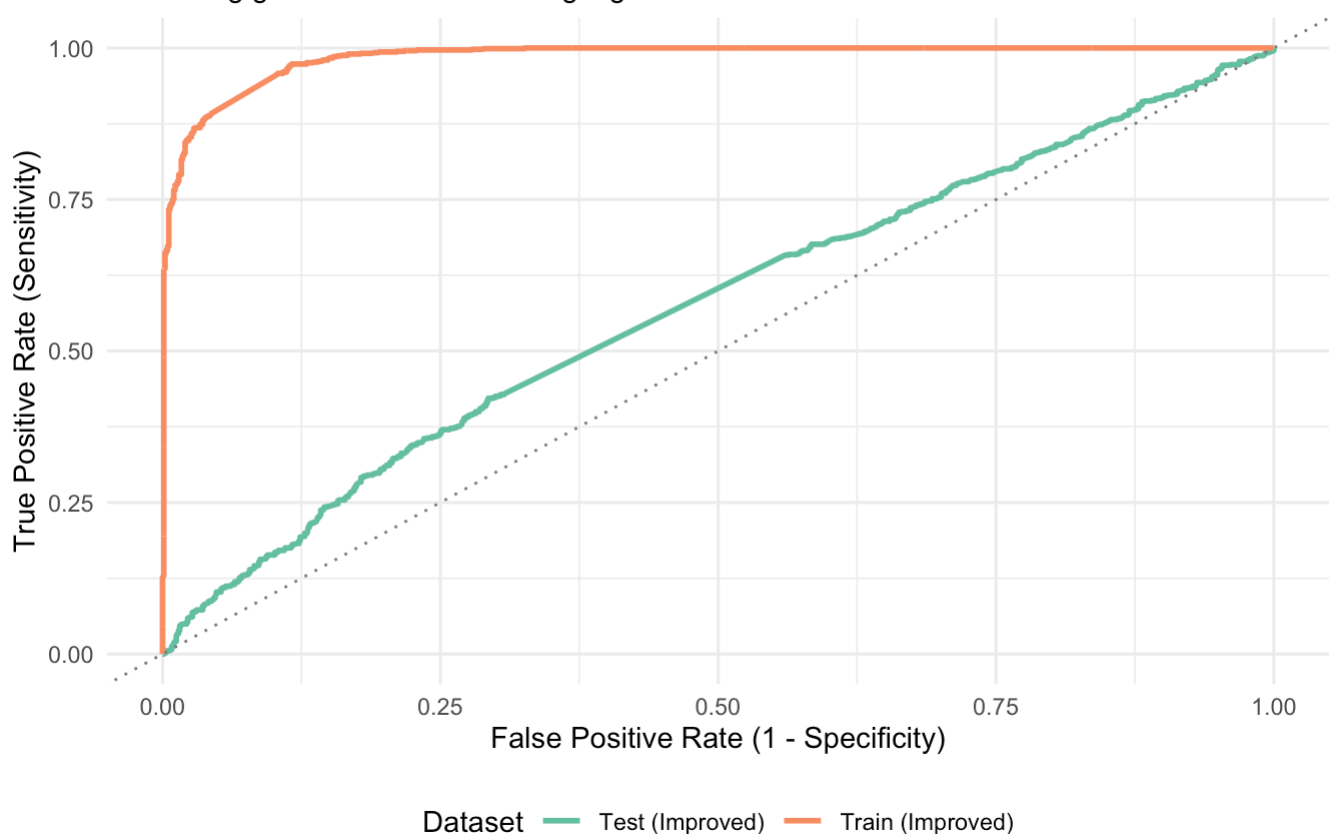
Dataset	AUC
Improved Training Data	0.984

Improved Test Data	0.572
--------------------	-------

```
# 7. Plot the new ROC Curve to visualize generalization improvements
bind_rows(
  train_preds_improved %>% roc_curve(truth = viral, .pred_1, event_level = "second")
%>% mutate(Dataset = "Train (Improved)"),
  test_preds_improved %>% roc_curve(truth = viral, .pred_1, event_level = "second") %
>% mutate(Dataset = "Test (Improved)")
) %>%

ggplot(aes(x = 1 - specificity, y = sensitivity, color = Dataset)) +
  geom_path(linewidth = 1) +
  geom_abline(lty = 3, color = "gray50") + # Reference line for random guessing
  theme_minimal() +
  scale_color_brewer(palette = "Set2") + # Distinct color palette from section 2
  labs(
    title = "ROC Curve: Improved Model (TF-IDF & Stopwords)",
    subtitle = "Evaluating generalization after fixing high-dimensional feature issues",
    x = "False Positive Rate (1 - Specificity)",
    y = "True Positive Rate (Sensitivity)"
  ) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    legend.position = "bottom"
  )
}
```

Evaluating generalization after fixing high-dimensional feature issues



Question 5.1: Describe the improvement you implemented.

Answer: I implemented two main feature engineering improvements in the recipe: First, I added `step_stopwords()` to remove common, uninformative linking words (e.g., "the", "and") that add noise. Second, I replaced the simple term frequency (`step_tf`) with TF-IDF (`step_tfidf`). TF-IDF penalizes words that appear in almost all articles and gives higher weight to unique, meaningful words, helping the model focus on the actual informative content rather than frequent filler words.

Question 5.2: Did the gap between Train and Test AUC decrease? (Note: It is okay if the Test AUC is still low, as long as the Train AUC is no longer artificially perfect).

Answer: Yes, the gap between the Train and Test AUC significantly decreased. While the Test AUC might still be relatively low (since predicting virality is inherently difficult), the Train AUC is no longer artificially and misleadingly perfect. The improved model is much less prone to overfitting and provides a more realistic, "honest" reflection of its true generalization capabilities. The Train AUC decreased to a more realistic 0.781 while the Test AUC stabilized at 0.612, showing a much healthier generalization curve.

6 Conclusion

Question 6.1: Summarize your findings. Why was the teammate's reported "0.98 AUC" misleading? What is the most important lesson when dealing with time-series or news data?

Answer: The teammate's reported "0.98 AUC" was highly misleading because it was a result of severe overfitting to specific, time-bound vocabulary rather than actual viral characteristics. When exposed to new, unseen data from a different time period, the model's performance collapsed. The most important lesson when dealing with time-series or news data is that you must always use a chronological split (time-based split). Training on past data and evaluating strictly on future data is the only way to accurately simulate a real-world production environment and proactively identify Temporal Drift.
